behavior of the model, essentially only testing that the controller makes a specific model call and using the model test to verify model behavior. Among the benefits of using mocks this way is that you're encouraged to make the interface between your controllers and models as simple as possible. However, it does mean that the controller test knows more about your model than it otherwise might, which may make the model code harder to change.

Also be careful of mocking methods that have side effects or that call other methods that might be interesting. The mock bypasses the original method, which means no side effect and no calling the internal method.

Pro tip: saving to the database and outputting to the response stream are both side effects.

Be cautious if you're specifying a value as a result of a mock and then asserting the existence of the very same value. One of the biggest potential problems with any test suite is false positives, and testing results with mocked values is an easy way to generate false positives!

Mocks Are Design Canaries

Test-driven development in general and mock objects in particular are sensitive indicators of your application code's quality. As the code becomes more complex and tightly coupled, the tests become harder and harder to write.

When you're using a true mock to encapsulate a test and isolate it from methods that are not under test, try to limit the number of methods you're mocking in one test. The more mocks, the more vulnerable the test will be to changes in the actual code. Having a lot of mocks may indicate that your test is trying to do too much, or it might point to a poor object-oriented design where one class is asking for too many details of a different class.

What You've Done

You've learned a lot about model testing—about the services Rails provides for testing models, and fixtures and factories as mechanisms for creating consistent test data. With this chapter you've started to transition from testing models to testing the user-facing parts of the application. Mock testing is useful for testing models, but it becomes especially useful when trying to shield the various layers of your application from each other.

In the next section you'll learn about testing the controller and view layers. Mock objects can be an important part of controller testing; creating mock models allows the controller tests to proceed independently of the model test.